MediCare Pro Hospital Management System

Red Team and Blue Team Security Analysis of Selected Web Attacks

SQL Injection · Reflected Cross-Site Scripting · Server-Side Template Injection

Red Team: Hernandez, Mark David · Ambong, Vincent Redd · Fernandez, Lance Cedrick · Nanoy, Joshua James · Paguio, Rafael Aaron
Blue Team: Penola, John Paul S. · Egil, Abdullah Ibn Mujahid · Espino, Luis David · Libag, Joshua

Submitted to: Engr. Jerry Boromeo | College of Computer Studies | October 2026

Context

MediCare Pro runs as a Node.js/Express staff portal with Firebase Auth, Cloud Firestore, and seven sidebar modules.

Website link: https://ias-6v7g.onrender.com/

Application

- **Demo hospital staff portal** built for security-learning.
  Node.js/Express server on port 3000; ships a Dockerfile for deployment.
  Browser stack is static HTML/CSS/JS with the Firebase SDK.

Data & modules

- **Firebase Authentication + Cloud Firestore;** falls back to in-memory simulated data.
  Sidebar: Patient Management, Staff Portal, Administration — plus dashboard and an AI chat widget.
  Seeded staff accounts carry weak, well-known lab credentials.

This presentation analyzes three web attacks and their fixes, with SSTI as the out-of-ordinary selection.

Overview

- **Three attacks analyzed:** SQL injection, reflected cross-site scripting (XSS), and server-side template injection (SSTI).
  Each attack is presented red team first — attack surface, exploitation, and impact — then blue team with the fix.
  SSTI is the out-of-ordinary choice: the most severe original exploit path, and the one already removed from the current codebase.
  All observations are drawn from the MediCare Pro source tree and its accompanying attack documentation.

SQL injection targets the Patient Records module through the Patient ID lookup.

Red Team · SQLi

- **Patient Records page** exposes a Patient ID field and a Search Records button, under Patient Management.
  The backend consults /api/sql?id=; the vulnerable lab concatenates raw input into the SQL text.
  Vulnerable pattern: const query = `SELECT … WHERE id = '${rawId}';
  A crafted single quote closes the string and lets the rest of the input become SQL syntax.

A SQL payload concatenates into the query and dumps every patient row.

Red Team · SQLi

- **Tautology payload** `1' OR '1'='1` bypasses the WHERE clause and returns all rows.
  The sequence `--` comments out the trailing quote the original query still expects.
  The semicolon `;` starts a new statement, which is what enables write operations in the lab.
  Baseline `1` returns one row (Maria Santos); the tautology returns all patients.

The UNION payload recovers plaintext credentials for every seeded staff account.

Red Team · SQLi

- **O' UNION SELECT username, password, role FROM users --** steals the lab users table.
  Recovers admin/admin123, doctor/password1, nurse/nurse123, and radiologist/xray2024 in cleartext.
  Write payloads can insert, update, delete rows, and even drop the table in the lab state.
  The engine is simulated (process memory); the attack logic is real against a real database.

This demo shows the SQL injection attack against the Patient Records module.

Demo · SQLi attack

DEMO VIDEO

SQL injection attack — Patient Records

1ST ATTACK

Show the Patient Records page, submit 1' OR '1'='1, then the UNION SELECT payload to recover credentials.

(replace this placeholder with your demo video link)

This demo shows how parameterizing the query stops the SQL injection attack.

Demo · SQLi

DEMO VIDEO

SQL injection solution — parameterized lookup

SOLUTION TO THE 1ST ATTACK

Show the secured api/sql.js query constant and the exact-match, escaped rendering on the Patient Records page.

(replace this placeholder with your demo video link)

Parameterizing the query binds input as data and stops SQL injection.

Blue Team · SQLi

- **The fix:** never build SQL by concatenating user input.
  The query text is a constant; the input is passed separately as a bound parameter.
  const QUERY = 'SELECT … WHERE id = ?'; params = [String(rawId)];
  1' OR '1'='1 is now matched as a literal string — and returns nothing.

The Patient Records page now matches input exactly and escapes every rendered cell.

Blue Team · SQLi

Server side

- **Query text is a constant;** input is bound as data, never parsed as SQL.
  No INSERT, UPDATE, DELETE, or DROP path exists in the secured file.

Client side

- **findPatients()** compares the term literally on id and patientId.
  renderResults() applies escapeHtml to every cell before inserting markup.

Reflected XSS targets the Patient Search module through the name reflection sink.

Red Team · Reflected XSS

- **Patient Search page** has a Search Patient Name field and a Search button.
  The sink is client-side: the search term is written to innerHTML under "Showing results for."
  The page also reads ?name= from the URL and auto-searches on load.
  A crafted link can fire the payload without any typing in the box.

A crafted search term executes in the victim’s browser through innerHTML.

Red Team · Reflected XSS

- **Payload** <img src=x onerror=alert(document.domain)> fires a script in the page.
  URL variant: /pages/xss-reflected.html?name=<img src=x onerror=alert(1)>
  Other variants also execute: <script>alert('XSS')</script> and “><svg/onload=alert(1)>.
  It can chain into the CSRF endpoint because that API trusts any origin.

This demo shows the reflected XSS attack against the Patient Search module.

Demo · XSS attack

DEMO VIDEO

Reflected XSS attack — Patient Search

2nd Attack

Show the Patient Search page, submit the img onerror payload, and then the URL-based auto-search variant.

(replace this placeholder with your demo video link)

This demo shows how textContent and a CSP stop the reflected XSS attack.

Demo · XSS

DEMO VIDEO

Reflected XSS solution — textContent + CSP

SOLUTION TO THE 2ND ATTACK

Show the reflected.textContent = input fix and the Content-Security-Policy header on responses.

(replace this placeholder with your demo video link)

Writing the reflected term with textContent renders it as text, not markup.

Blue Team · XSS

- **The fix:** treat untrusted input as text, not as markup.
  Replace reflected.innerHTML = input with reflected.textContent = input.
  The browser inserts the string as plain text; scripts and HTML do not execute.
  This neutralizes both the box payload and the URL–based auto–search variant.

A strict Content Security Policy blocks unauthorized script execution as defense-in-depth.

Blue Team · XSS

Policy

- **Add a CSP header:** default-src 'self'; script-src 'self'.
  The browser loads resources only from the app's own origin.

Why it helps

- **It refuses** unauthorized inline scripts if a markup sink is reintroduced.
  Pair XSS fixes with proper CSRF protection so a script cannot abuse the victim's session.

SSTI targets the Patient Feedback module and is the most severe, now-removed path.

Red Team · SSTI

- **Patient Feedback page** has a Patient ID, a rating, and a Comments field.
  The original lab built the report as a template and evaluated every ${…} with new Function.
  The escaping looked real but was only String() — security theater, not a defense.
  SSTI is the out-of-ordinary choice: most severe original path, and already removed.

The chain escalates from a single comment box to file read and OS command execution.

Red Team · SSTI

Reconnaissance

- **${7*7}** returns Comments: 49 — confirms server-side evaluation. **${Object.keys(process.env).join(', ')}** lists every environment variable name.

Exploitation

- **${process.env.FIREBASE_SERVICE_ACCOUNT}** steals the Firebase admin credential. **${process.mainModule.require('child_process').execSync('whoami')}** returns root.

This demo shows the SSTI attack against the Patient Feedback module.

Demo · SSTI attack

DEMO VIDEO

SSTI attack — Patient Feedback

3RD ATTACK

3RD ATTACK PART 2

Show ${7*7} → 49, the environment-variable name dump, and the service-account credential exfiltration.

Demo video: SSTI solution — SOLUTION TO THE 3RD ATTACK

Conclusions

SSTI is closed by removing the evaluator and keeping comments as data — and the same rules defensively contain the other two attacks.

- **SQL injection:** parameterize queries, match input as data, and escape every rendered cell.
  Reflected XSS: write untrusted input with textContent and add a strict Content Security Policy; fix CSRF too.
  SSTI: remove the template evaluator; keep user comments as plain data, never as code.
  Defense-in-depth across all three: least privilege, no driver errors to clients, and authentication on every route.
  State scope honestly: SQL injection is simulated in the current codebase; SSTI is removed.

THANK YOU FOR LISTENING!